



**Instituto Tecnológico Superior de
Ciudad Constitución.**

**INGENIERÍA EN SISTEMAS
COMPUTACIONALES.**

Ciencia de datos

“Practica ETL part1”

Instructor: Isaac Felipe Garcia Lopez

Grupo: 8SM

Alumno: Luis Manuel Muñoz Lizarraga

Fecha de entrega: 08 - 06 - 2026

Cd. Constitución, B.C.S.

Introducción

En esta práctica se realizó un proceso ETL utilizando Python. ETL significa extracción, transformación y carga de datos. La finalidad fue tomar la información del archivo `members.csv`, limpiar y transformar algunos campos importantes, y finalmente guardar el resultado en una base de datos SQLite. Este tipo de proceso es común en ciencia de datos, análisis de información y preparación de datos para proyectos posteriores.

El reporte explica paso a paso cómo se creó el ambiente virtual llamado `mlearning`, qué herramientas se utilizaron y qué técnicas se aplicaron en el código `data_practica.py`. También se explican las expresiones regulares utilizadas para extraer el código postal, limpiar el número telefónico y trabajar con fechas.

Objetivos de la práctica

- Crear un ambiente virtual de Python llamado mlearning para trabajar de forma ordenada.
- Utilizar el archivo members.csv como fuente de datos para el proceso ETL.
- Aplicar expresiones regulares para extraer el código postal desde la columna address.
- Limpiar el número telefónico eliminando símbolos como +, paréntesis de apertura y paréntesis de cierre.
- Dar formato a la fecha de nacimiento en día, mes y año.
- Calcular la edad en años tomando como referencia la columna register_time.
- Guardar el resultado transformado en una base de datos SQLite.

Conceptos utilizados

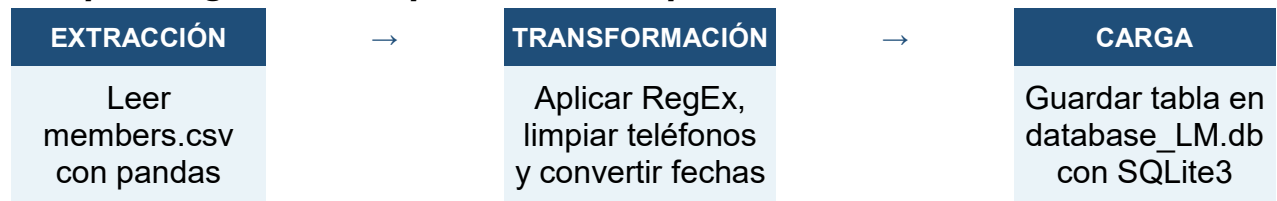
3.1. ¿Qué es un ambiente virtual?

Un ambiente virtual permite crear un espacio separado para instalar las librerías de un proyecto sin afectar otros proyectos de Python. En este caso, el ambiente se llamó mlearning. La documentación oficial de Python explica que el módulo venv sirve para crear ambientes virtuales ligeros con sus propios paquetes instalados de forma aislada.

3.2. ¿Qué es ETL?

ETL es un proceso compuesto por tres fases: extracción, transformación y carga. Primero se obtienen los datos desde una fuente, después se limpian o modifican, y al final se guardan en un destino, por ejemplo una base de datos. En esta práctica, la extracción se hizo desde un archivo CSV, la transformación se realizó con Pandas y RegEx, y la carga se hizo en SQLite3.

Esquema general del proceso ETL aplicado



Herramientas utilizadas

Herramienta	Uso en la práctica	Explicación
Python	Lenguaje principal	Se utilizó para ejecutar el proceso completo de extracción, transformación y carga.
Pandas	Manejo de datos	Sirvió para leer el CSV, crear el DataFrame y transformar columnas.
RegEx	Patrones de texto	Se usó para detectar códigos postales y símbolos dentro del teléfono.
SQLite3	Base de datos local	Permitió crear una base de datos y guardar la tabla transformada.
GitHub	Repositorio	Se incluyó el enlace del repositorio personal de la práctica.

Creación del ambiente virtual mlearning

Para trabajar de manera ordenada, se crea un ambiente virtual llamado mlearning. Esto ayuda a separar las librerías del proyecto y evita problemas con versiones instaladas en otros trabajos.

Paso / comando	Descripción
<code>python -m venv mlearning</code>	Crea el ambiente virtual con el nombre mlearning.
<code>mlearning\Scripts\activate</code>	Activa el ambiente virtual en Windows.
<code>source mlearning/bin/activate</code>	Activa el ambiente virtual en Linux o macOS.
<code>pip install pandas</code>	Instala Pandas dentro del ambiente virtual.
<code>python data_practica.py</code>	Ejecuta el archivo de la práctica.

deactivate	Cierra o desactiva el ambiente virtual cuando se termina de trabajar.
------------	---

SQLite3 no requiere una instalación externa en la mayoría de las instalaciones de Python, ya que Python incluye un módulo sqlite3 para trabajar con bases de datos SQLite mediante una interfaz compatible con DB-API 2.0.

Explicación del código data_practica.py

El código inicia importando las librerías necesarias. Pandas se usa para trabajar con el archivo CSV y sqlite3 se usa para crear la base de datos local donde se almacena el resultado transformado.

```
import pandas as pd
import sqlite3
```

Extracción de datos

La extracción consiste en leer el archivo members.csv. En el código se utiliza una URL en formato raw de GitHub para cargar el archivo directamente con pd.read_csv. Si ocurre un error durante la lectura, el programa lo muestra en pantalla y termina la ejecución.

```
csv_url = "https://raw.githubusercontent.com/NationalSpark37/Practica-ETL-part1/refs/heads/main/members%20(2).csv"

try:
    df = pd.read_csv(csv_url)
except Exception as e:
    print(f"Error de lectura: {e}")
    return
```

Uso de RegEx en la práctica

Las expresiones regulares, también llamadas RegEx, permiten buscar patrones dentro de textos. La documentación oficial de Python explica que una expresión regular especifica un conjunto de cadenas que coinciden con un patrón. En esta práctica se usaron para localizar códigos postales y para eliminar símbolos del número telefónico.

Patrón	Dónde se usa	Qué hace
(\\b\\d{5}\\b)	address → zip_code	Busca exactamente 5

		dígitos continuos como código postal.
[\\+\\(\\)]	phone_number	Busca los símbolos +, (y) para eliminarlos del teléfono.
%d-%m-%Y	birth_date	Muestra la fecha en formato día-mes-año.
unit='s'	register_time	Interpreta register_time como tiempo Unix en segundos.

Función para extraer el código postal

La primera transformación crea una nueva columna llamada zip_code a partir de la columna address. Para esto se usa str.extract, que en Pandas permite extraer grupos capturados por una expresión regular dentro de una serie de texto.

```
def extraer_codigo_postal(df, col_origen, col_destino):
    patron_cp = r'(\b\d{5}\b)'
    df[col_destino] = df[col_origen].str.extract(patron_cp)
    return df
```

El patrón busca cinco números seguidos. Los límites de palabra ayudan a evitar que se tome una parte de un número más largo. Así, si una dirección contiene un código postal como 23000, ese dato se extrae y se coloca en la nueva columna.

6.4. Función para limpiar el número telefónico

La segunda transformación limpia la columna phone_number. El objetivo es quitar caracteres que no son necesarios para el análisis, como el signo + y los paréntesis. Para esto se usa str.replace con regex=True. Según la documentación de Pandas, str.replace permite reemplazar ocurrencias de un patrón o expresión regular dentro de una serie.

```
def limpiar_telefono(df, col):
    patron_limpieza = r'[\\+\\(\\)]'
    df[col] = df[col].str.replace(patron_limpieza, '', regex=True)
    return df
```

El patrón `[\\+\\(\\)]` funciona como una lista de caracteres que se deben buscar. En este caso, cada vez que el programa encuentra un `+`, un `(` o un `)`, lo reemplaza por una cadena vacía, es decir, lo elimina.

6.5. Función para formatear la fecha de nacimiento

La tercera transformación convierte la columna `birth_date` a un formato más claro: día, mes y año. Para esto se usa `pd.to_datetime`, que convierte los valores a tipo fecha, y después `strftime` para mostrar el resultado en el formato deseado.

```
def formatear_fechas(df, col):
    df[col] = pd.to_datetime(df[col], format='mixed', errors='coerce').dt.strftime('%d-%m-%Y')
    return df
```

La opción `errors='coerce'` ayuda a que, si un dato no se puede convertir correctamente, Pandas lo transforme en un valor nulo de fecha. Esto evita que el programa se detenga por un error de formato.

6.6. Función para calcular la edad con register_time

La cuarta transformación calcula la edad en años. El código convierte `register_time` como una fecha basada en segundos Unix. Después convierte `birth_date` a fecha y calcula la diferencia entre ambas fechas. El resultado se divide entre 365 para obtener una edad aproximada en años.

```
def calcular_edad(df, col_nac, col_reg, col_edad):
    fecha_reg = pd.to_datetime(df[col_reg], unit='s', errors='coerce')
    fecha_nac = pd.to_datetime(df[col_nac], format='%d-%m-%Y', errors='coerce')

    df[col_edad] = (fecha_reg - fecha_nac).dt.days // 365
    df[col_reg] = fecha_reg.dt.strftime('%d-%m-%Y %H:%M:%S')
    return df
```

Además de calcular la edad, la función también formatea `register_time` para que se vea como una fecha completa con hora, minutos y segundos.

6.7. Carga de los datos transformados en SQLite3

Después de transformar los datos, el programa crea una base de datos llamada `database_LM.db` y guarda el `DataFrame` en una tabla llamada

miembros_transformados. La función to_sql permite enviar los datos de Pandas a una tabla de base de datos conectada con SQLite.

```
db_name = "database_LM.db"
conn = sqlite3.connect(db_name)
df.to_sql('miembros_transformados', conn, if_exists='replace', index=False)
conn.close()
```

La opción if_exists='replace' significa que, si la tabla ya existe, se reemplaza con la nueva versión. Esto es útil en prácticas porque permite ejecutar el código varias veces sin crear tablas duplicadas.

Orden completo del proceso ETL

Orden	Función / acción	Entrada principal	Resultado esperado
1	pd.read_csv(csv_url)	members.csv	DataFrame original cargado en memoria
2	extraer_codigo_postal()	address	Nueva columna zip_code
3	limpiar_telefono()	phone_number	Teléfono sin + ni paréntesis
4	formatear_fechas()	birth_date	Fecha en formato dd-mm-aaaa
5	calcular_edad()	birth_date y register_time	Nueva columna edad_registro
6	to_sql()	DataFrame transformado	Tabla final en SQLite

Técnicas aplicadas en el código

Las técnicas principales aplicadas en esta práctica fueron las siguientes:

- **Lectura de CSV:** Se usó `pd.read_csv` para importar los datos desde el archivo `members.csv` almacenado en GitHub.
- **Transformación con RegEx:** Se usaron patrones para extraer el código postal y para limpiar caracteres del número telefónico.
- **Manejo de fechas:** Se aplicó `pd.to_datetime` para convertir texto o números en fechas manejables por Pandas.
- **Cálculo de edad:** Se restó la fecha de nacimiento a la fecha de registro para calcular la edad aproximada en años.
- **Carga a SQLite:** Se creó una conexión con `sqlite3` y se guardó el resultado en una tabla llamada `miembros_transformados`.

Resultado esperado

Al ejecutar correctamente el archivo `data_practica.py`, el programa debe mostrar el mensaje “ETL Parte 1 completado con éxito”. También debe crear una base de datos SQLite llamada `database_LM.db`. Dentro de esta base de datos se guarda una tabla llamada `miembros_transformados` con las columnas originales más las columnas o datos transformados.

Elemento	Resultado esperado
<code>zip_code</code>	Código postal extraído desde la dirección.
<code>phone_number</code>	Número telefónico sin símbolos +, (y).
<code>birth_date</code>	Fecha de nacimiento en formato día-mes-año.
<code>register_time</code>	Fecha de registro convertida a formato legible.
<code>edad_registro</code>	Edad aproximada en años al momento del registro.
<code>database_LM.db</code>	Archivo de base de datos generado por SQLite.